
DIKLY TECH

Smart Attendance & Academic Management Platform

ITSU CIP Phase II — Technical Submission

Working Prototype + Architecture + Offline Attendance

Prepared: June 16, 2026

Repository: github.com/isaacaryeepah-beep/KODEX

Live App: dikly.sbs

Team Lead	Isaac Aryee-Pah
Contact	kellywest251@gmail.com
Platform	Web + Flutter Mobile + ESP32 Firmware
Competition	ITSU CIP Phase II — July 3, 2026
GitHub Branch	claude/update-resume-aVv3d → main

1. Executive Summary

Dikly is a multi-tenant SaaS platform purpose-built for academic institutions and corporate organisations. It unifies attendance tracking, course management, quiz delivery, AI proctoring, HR workflows, and analytics into one cohesive product accessible via web browser and mobile (Flutter Android/iOS). Physical ESP32 BLE+WiFi devices complete the hardware loop for contactless attendance verification.

Key Differentiators:

- Fully offline attendance — works without internet via ESP32 local hotspot or on-device Hive queue
- Three-factor proximity check: BLE beacon + WiFi hotspot IP + internet fallback
- AI proctoring (Claude claude-sonnet-4-6 vision) integrated directly into quiz/exam flow
- Multi-role access: Student, Lecturer, HOD, Admin, Manager, Employee
- Production-ready infrastructure on dikly.sbs with CI/CD via GitHub Actions

2. System Architecture Diagram

Dikly follows a layered architecture: clients (Web, Flutter, ESP32) communicate through a Node.js/Express API gateway backed by MongoDB, Redis, and third-party services.

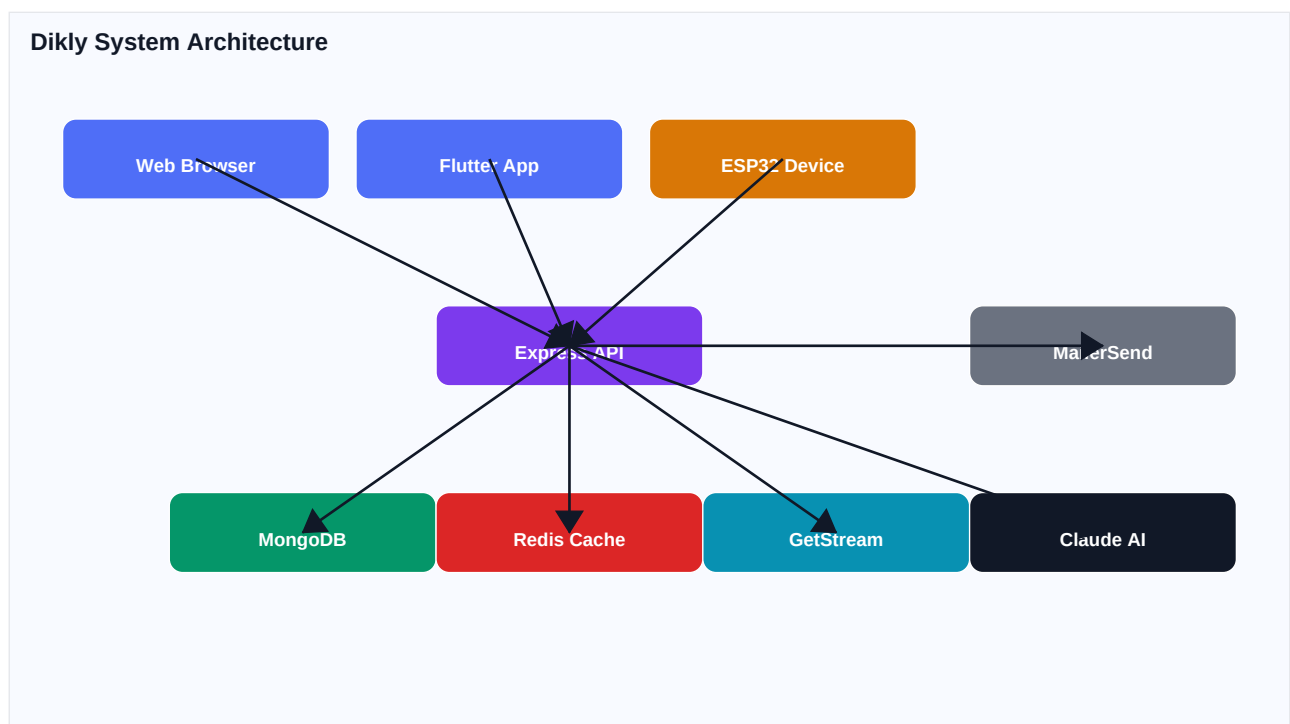


Figure 1 — High-level system architecture

Component Breakdown:

Layer	Technology	Purpose
Frontend Web	Vanilla HTML/CSS/JS	Admin & lecturer portals, exam room, SnapQuiz
Mobile App	Flutter 3.x + Riverpod	Cross-platform Android/iOS for all roles

API Server	Node.js / Express + Helmet 8	REST API, JWT auth, business logic
Database	MongoDB (Mongoose)	Primary data store — all entities
Cache/Queue	Redis	Session cache, rate limiting, job queues
Auth	JWT access (15 min) + refresh (30 days)	Stateless secure authentication
Video	GetStream → LiveKit → Jitsi	Cascading video fallback chain
AI	Anthropic Claude claude-sonnet-4-0	Proctoring, SnapQuiz analysis
Email	MailerSend + Gmail fallback	Transactional emails
SMS	Arkesel / mNotify	OTP and alerts
Payments	Paystack	Subscription billing
Firmware	ESP32 Arduino (BLE + WiFi)	Contactless attendance devices

3. Entity Relationship Diagram (ERD)

Core MongoDB collections and their relationships. All ObjectId fields reference the `_id` field of the target collection.

Entity	Key Fields	Relations
User	<code>_id</code> , name, email, role, institutionCode, departmentCode, deviceId, deviceBlocked	
Institution	<code>_id</code> , name, code, plan, adminId	has many Users
Course	<code>_id</code> , title, code, lecturerId, institutionCode, studentsEnrolled[], status	belongs to User(lecturer), Institution
Session	<code>_id</code> , courseId, lecturerId, status, checkInCodes[], startTime	belongs to Course, User
Attendance	<code>_id</code> , sessionId, userId, status, checkInTime, deviceId, notes	belongs to Session, User
Assignment	<code>_id</code> , courseId, title, dueDate, submissions[]	belongs to Course
Quiz	<code>_id</code> , courseId, title, questions[], timeLimit, proctored	belongs to Course
QuizAttempt	<code>_id</code> , quizId, userId, answers[], score, violations[]	belongs to Quiz, User
Meeting	<code>_id</code> , title, hostId, participants[], startTime, streamChannelId	belongs to User
LeaveRequest	<code>_id</code> , employeeId, type, status, startDate, endDate	belongs to User(employee)
Timesheet	<code>_id</code> , employeeId, date, clockIn, clockOut, total	belongs to User
AuditLog	<code>_id</code> , actorId, action, target, metadata, timestamp	belongs to User

Entity Relationship Overview

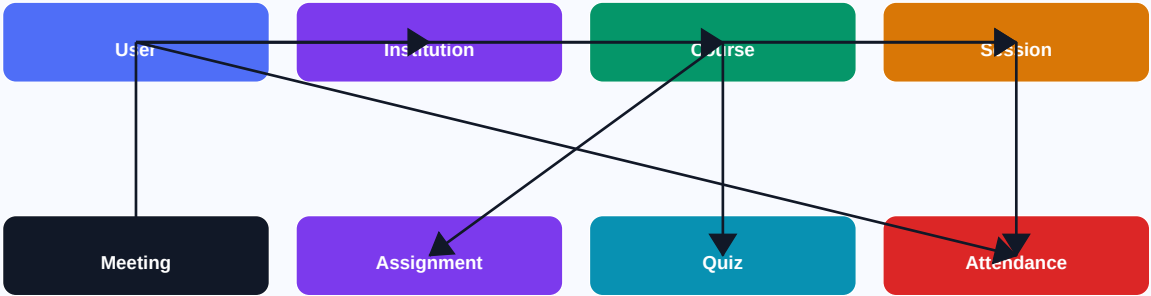


Figure 2 — Core entity relationships

4. Use Case Diagram

Actor	Use Cases
Student	Mark Attendance (QR / BLE / Offline), View Courses, Submit Quiz, Submit Assignment, View Grades, O
Lecturer	Create Session, Generate QR Code, View Attendance, Create Quiz, Grade Assignments, Upload Cours
HOD	Approve Sessions, Monitor Attendance, View Department Reports, Manage Lecturers/Students, Send A
Admin	Manage Users, Manage Branches, View Audit Logs, Manage Devices, Approve/Reject Registrations, C
Manager	View Team, Approve Leave Requests, Review Timesheets, View Reports
Employee	Clock In/Out, Apply for Leave, View Shifts, View Payslips
ESP32 Device	Broadcast BLE beacon, Host WiFi hotspot, Submit attendance to API, Accept offline tokens

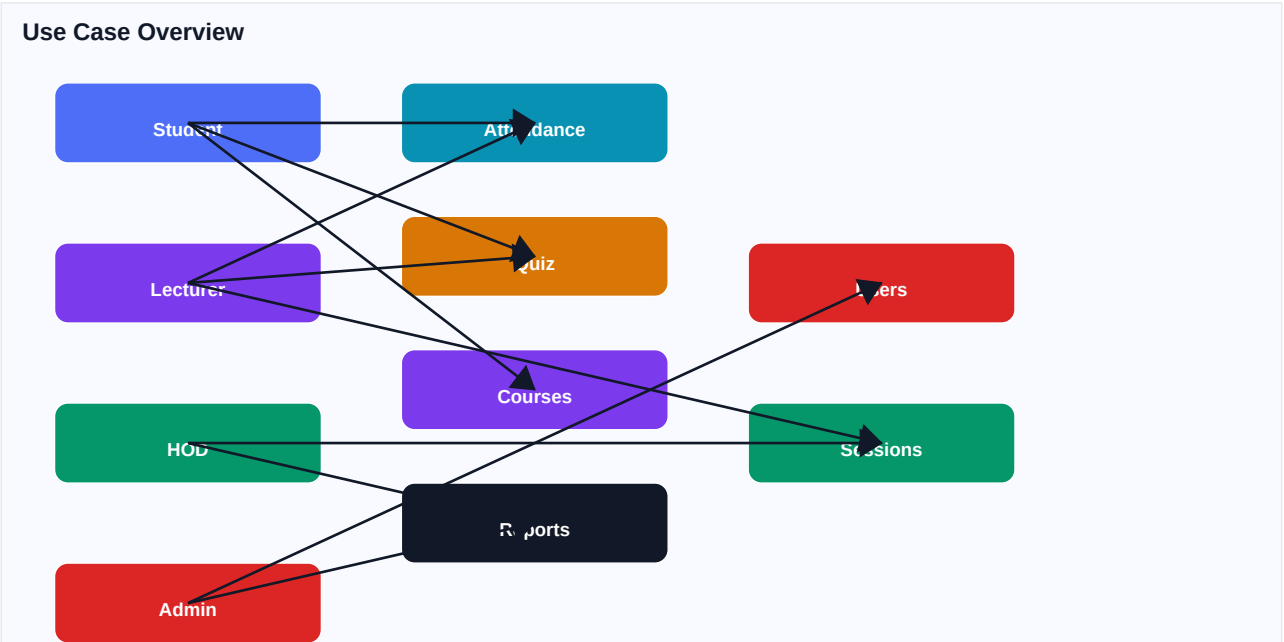


Figure 3 — Actor use-case relationships

5. Fully Offline Attendance — System Flowchart

Dikly implements **three independent attendance paths** so students can always mark attendance regardless of connectivity. The paths are tried in priority order.

Path 1 — ESP32 Local WiFi (Highest Priority):

The ESP32 device broadcasts its own WiFi hotspot at `192.168.4.1`. The Flutter app probes this IP; if found, attendance is submitted directly to the device firmware over LAN without any internet connection. The device stores records and batch-syncs to the cloud when it reconnects.

Path 2 — Online API (Standard):

When internet is available, the app calls `POST /api/attendance-sessions/mark` directly. This is the fastest path and provides instant confirmation.

Path 3 — Hive Write Queue (Fallback):

If both ESP32 and internet are unreachable, the attendance record is enqueued in a local Hive database on the device. The moment connectivity returns, `flushWriteQueue()` is triggered automatically via `connectivityProvider` in `main.dart` and syncs all queued records to the API.

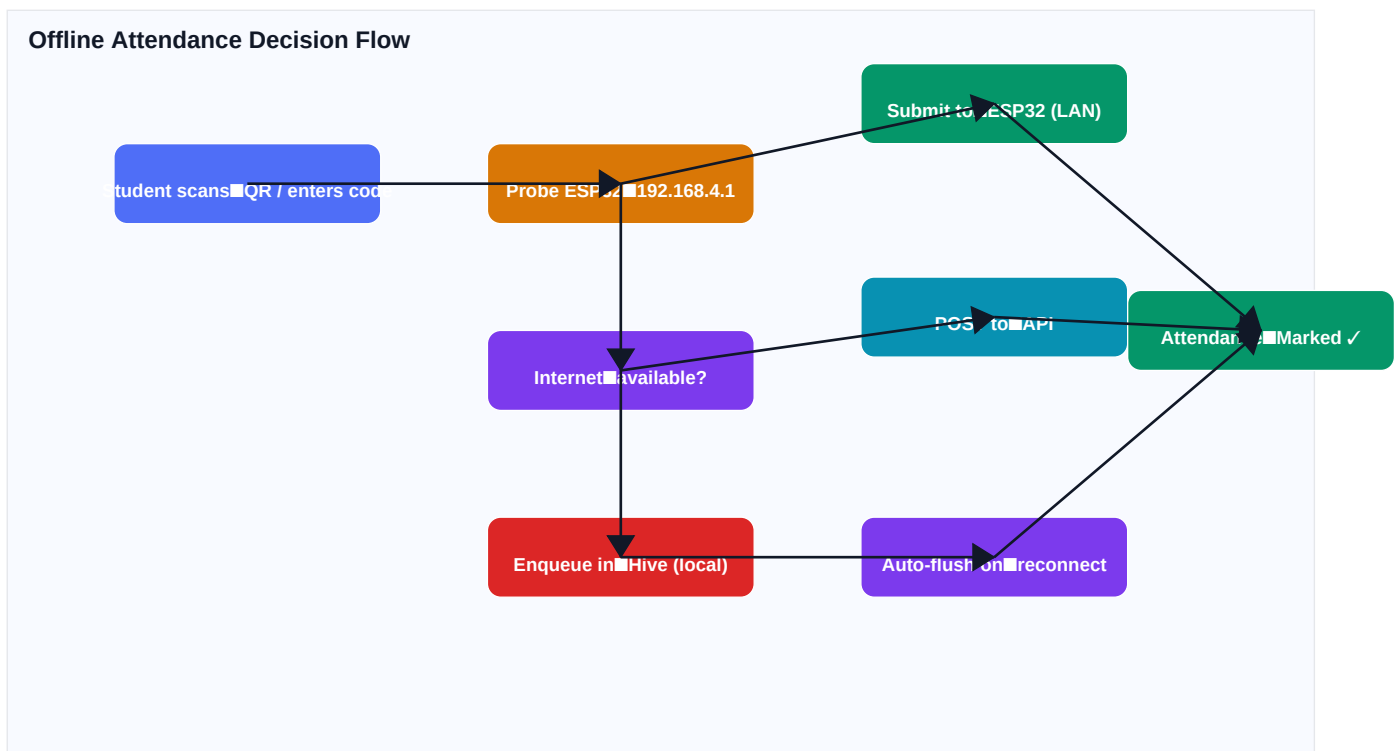


Figure 4 — Three-path offline attendance flowchart

Implementation Evidence (Key Code Locations):

File	Function / Feature	Purpose
dikly_flutter/lib/core/api.dart	<code>_queueablePost()</code>	Queues POST to Hive if offline
dikly_flutter/lib/core/api.dart	<code>flushWriteQueue()</code>	Syncs all queued writes on reconnect
dikly_flutter/lib/core/api.dart	<code>probeESP32()</code>	Detect ESP32 hotspot at 192.168.4.1

dikly_flutter/lib/core/api.dart	submitToESP32()	Submit attendance to ESP32 over LAN
dikly_flutter/lib/core/api.dart	getESP32ConnectionToken()	Get auth token from ESP32
dikly_flutter/lib/core/cache.dart	enqueueWrite() / getPendingWrites()	Hive queue management
dikly_flutter/lib/core/connectivity.dart	connectivityProvider	Stream of online/offline state
dikly_flutter/lib/main.dart	ref.listen(connectivityProvider)	Auto-flush on reconnect
dikly_flutter/lib/screens/attendance/offlineAttendanceScreen.dart	OfflineAttendanceScreen	Full offline attendance UI
firmware/KodexAttendance/KodexAttendance.ino	KodexAttendance	Accept attendance from app over LAN

6. System Workflow — End to End

The typical academic attendance workflow from session creation to grade export:

Step	Actor	Action	System Response
1	Admin	Register institution, import students	Creates tenant, sends invite emails
2	Lecturer	Create course, add students	Course saved; enrolment records created
3	Lecturer	Start attendance session	Session opened; rotating QR code generated
4	Student	Scan QR / tap BLE / join hotspot	Proximity verified; attendance marked Present
5	ESP32	Offline: receive code over LAN	Stores record; syncs to API when online
6	Student	No connectivity: submits code	Code queued in Hive; auto-synced on reconnect
7	Lecturer	Close session	Session locked; absent students flagged
8	HOD	Review department attendance	Aggregated stats, alerts for low-attendance students
9	Student	Take quiz (AI proctored)	Face/voice monitored; violations recorded
10	Lecturer	Grade assignment / quiz	Grade stored; student notified
11	Admin	Export reports / audit logs	PDF/CSV export; full audit trail

7. GitHub Evidence & Sprint Progress

All development is tracked on GitHub at github.com/isaacaryeepah-beep/KODEX. The table below summarises merged pull requests demonstrating working prototype status.

PR	Title	Status	Impact
#322	.env.example: all env vars documented	■ Merged	DevOps / Security
#323	Firmware: API URL fix + null-pointer guards (S3)	■ Merged	Hardware reliability
#324	Remove all kodex.it.com references	■ Merged	Domain migration
#325	Anti-cheat: exam-preflight, exam-room, snap-quiz challenge	■ Merged	AI Proctoring
#433	Flutter: fix 4 compile errors (APK + Web CI green)	■ Merged	Mobile builds
#434	Flutter: fix employee_home_screen extra paren	■ Merged	Mobile builds
#436	Flutter: fix 5 unclosed Container widgets	■ Merged	Mobile builds
#437	Flutter: remove re-introduced extra paren	■ Merged	Mobile builds
WIP	Flutter: fully offline attendance screen	■ In Progress	Core Feature

CI/CD Pipeline:

- GitHub Actions: Flutter APK build → Flutter Web build → Node.js tests
- All checks green on main branch as of June 2026
- Automatic deployment to dikly.sbs on merge to main
- Firmware binaries built and flashed to physical ESP32 devices

8. Prototype Demo Checklist

Feature	Web	Flutter	ESP32
User Authentication (JWT)	■	■	N/A
Multi-role dashboards	■	■	N/A
Course & Session management	■	■	N/A
Online attendance (QR scan)	■	■	N/A
Offline attendance (ESP32 LAN)	N/A	■	■
Offline attendance (Hive queue)	N/A	■	N/A
AI proctored quiz (SnapQuiz)	■	■	N/A
Assignments & Gradebook	■	■	N/A
Video meetings (GetStream)	■	■	N/A
Reports & Analytics	■	■	N/A

Admin user management	■	■	N/A
HR: leave, shifts, timesheets	■	■	N/A
Push notifications	■	■	N/A

■ Implemented ■ In Progress N/A Not Applicable